

the performance with the 'locality of reference'. However, this is not always possible if, for instance, the preferred or ideal processor is busy for further scheduling. The scheduler then migrates the process to a different processor for execution.

On the other hand, hard affinity provides users/programmers the flexibility to override the natural affinity for their tasks/processes. In Linux, all processes are represented by the kernel data structure *task_struct* that contains fields related to the process attribute. Among these is the *cpus_allowed* bitmask field that specifies which CPU(s) shall handle the task. This bitmask consists of a series of *n* bits, one for each *n* logical processor in the system. So if a system has four processors (i.e., a multi-processor system), this bitmask will have four bits, and if each processor is a dual core, then it'll have an eight-bit bitmask.

The default state of a process in Linux for the *cpus_allowed* field is all 1s. It indicates that the process is allowed to run on any available CPU, and can migrate across processors as and when required.

Hard affinity allows you to alter this bit field. The scheduler then honours it and schedules your task on the processor of your choice. We will soon look at how you, as a user/programmer, can change the affinity.

But let's first discuss the possible reasons that may lead you to design your application to override the natural (or soft) affinity of the process/thread.

Why overrule a natural affinity?

Well, the Linux scheduler does a fantastic job of scheduling. It tries to run a process on the same processor it ran the last time, assuming that some remnants of the process may be left (especially the cache) and thus a better performance could be achieved. However, there are various other parameters that a scheduler considers while deciding which processor should run the process.

One of the possible reasons could be that the preferred processor is busy but the other processor(s) in the system is not. Under the circumstances, the scheduler will dispatch the process to the idle processor in order to maintain the load balance.

However, for whatever reason, a program might want to have control over the scheduling aspects of the application. Some of these could be:

- A multi-processor system needs to keep the processor's caches valid. Data must be modified by only one processor, and all other processors that have cached the same data must invalidate their copy and fetch the most recent data again in case of a cache miss, and they do so. This may come at a high cost in terms of performance. Now think of a situation when a process starts bouncing between different processors. This will constantly cause the cache to get invalidated. And the situation may even worsen, if the threads of a process are scheduled at discrete processors and they are perpetually accessing and updating the same piece of data. This will lead to the frequent invalidation of the cached data. Here,

hard affinity will rescue you from such performance degradation by letting you schedule your application on the processor(s) of your choice.

- In NUMA (Non-Uniform Memory Architecture) machines, processors will have faster access to local memory than shared memory between different processors. Therefore, forcing a process to the processor that has local access to the frequently used memory helps in boosting the performance.
- Sometimes real-time applications require a dedicated processor. With hard affinity you can ensure that a long-running and time-sensitive application runs on a specific processor.

Linux kernel 2.6 provides complete control to set and retrieve the CPU affinity of a process. However, a word of caution before we proceed: using hard affinity might cause the processors to have uneven loads.

CPU affinity—a user's perspective

To set or retrieve the CPU affinity of a running process from the shell prompt, you can use the *taskset* command, with which you can even launch a new task with a given affinity. Let's see how we can do that.

Let's suppose we have a running process, with PID (process ID) 21934. To bind this process to Processor #0 (the processor count starts from 0), let's issue the following command:

```
$ taskset -p 0x01 21934
```

Here, the *-p* flag indicates that *taskset* operates on an existing process. The hex value *0x01* tells the new affinity mask of the process (i.e., CPU #0). Finally, the third parameter 21934 is the PID of the task.

To test the above example, you first need to find the PID of an already running process using the *ps* command. In my case, the process I picked had a PID number of 21934. Here's the output after running the command:

```
pid 21934's current affinity mask: 2
```

```
pid 21934's new affinity mask: 1
```

From this output, we can conclude that for the process with PID 21934, the affinity mask is reset from 2 to 1. Now let's explore how the current affinity mask of a running process can be seen (in my case, I picked out a running process with PID 21934):

```
$ taskset -p 21934
```

```
pid 21934's current affinity mask: 1
```

You can also check the CPU affinity list (i.e., the list of all the processors that can run the process) of a process. This can be done with *-c* flag. Let's have a look at it.

```
$ taskset -c -p 21230
```